

ΥΠΟΛΟΓΙΣΤΙΚΗ ΝΟΗΜΟΣΥΝΗ

ΕΡΓΑΣΙΑ 2: ΤΕΧΝΗΤΑ ΝΕΥΡΩΝΙΚΑ ΔΙΚΤΥΑ



ΔΗΜΟΚΡΙΤΕΙΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΡΑΚΗΣ

ΤΜΗΜΑ
ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ
& ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Ιατρού Κωνσταντίνος 58071

Άσκηση 1 (Πρόβλημα προσέγγισης συνάρτησης)

ΠΙΝΑΚΑΣ ΠΕΡΙΕΧΟΜΕΝΩΝ

Α. Εισαγωγή.....	2
Β. Αλγόριθμος Επίλυσης	3
Γ. Σημειώσεις.....	18

A. ΕΙΣΑΓΩΓΗ

Στα πλαίσια της δεύτερης εργασίας ζητείται να εκπαιδευτεί νευρωνικό δίκτυο που να προσεγγίζει την παρακάτω συνάρτηση 2 μεταβλητών:

$$f(x, y) = 0.7e^{\cos(\pi x)} + 0.3 \cos(2\pi y), \quad x, y \in [-4, 4]$$

Ζητήθηκε να χρησιμοποιηθούν 400 δείγματα ως σύνολο δεδομένων εκπαίδευσης.

Αρχικά θα γίνει η εκπαίδευση του νευρωνικού δικτύου μέσω του γραφικού περιβάλλοντος που δίνεται από το MATLAB, nprtool.

Στην συνέχεια θα γραφεί script που να εκπαιδεύει νευρωνικό δίκτυο με όμοια δομή με αυτή που παρέχει το nftool και θα τυπώσουμε διαγράμματα (πραγματικό και δικτύου) με τυχαίες τιμές των x και y (για να μην είναι ίδιες με αυτές των δειγμάτων) που να ανήκουν στο διάστημα $[-4, 4]$.

Τέλος θα γραφεί ένα script βασισμένο στο προηγούμενο, αλλά με βελτιώσεις που δεν μπορούν να γίνουν μέσω του γραφικού περιβάλλοντος (πχ περισσότερα κρυφά επίπεδα).

B. ΕΠΙΛΥΣΗ

1.NPRTOOL

Αρχικά δημιουργώ τα δείγματα μέσω του command window:

```
>> x = round([-4:0.421:4],2);  
>> y = x;  
>> [X, Y] = meshgrid(x, y);  
>> F = 0.7.*exp(cos(pi.*X))+0.3.*cos(2*pi.*Y);  
>> x = reshape(X, [1 numel(X)]);  
>> y = reshape(Y, [1 numel(Y)]);  
>> T = reshape(F, [1 numel(F)]);  
>> P = [x;y];
```

Τα διανύσματα x και y έχουν τιμές από -4 μέχρι 4, οι στρογγυλοποίηση γίνεται για να χρησιμοποιηθούν στα δείγματα τα άκρα του διαστήματος.

Στην συνέχεια ορίζω του πίνακες X και Y του meshgrid για να δημιουργήσω των πίνακα με τις αντίστοιχες τιμές F για την κάθε x και y τιμή.

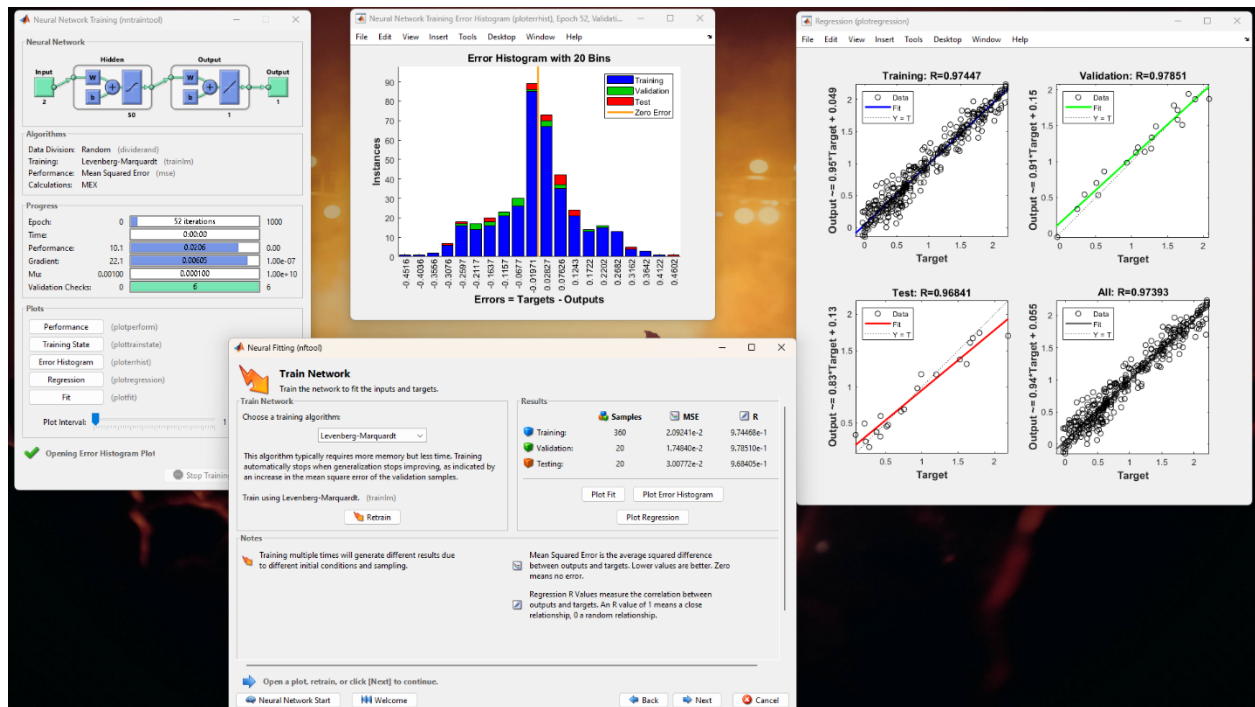
Πριν δημιουργήσω τον πίνακα P με διαστάσεις 2x400 που θα έχει τα δείγματα των εισόδων, πρέπει να κάνει reshape τους πίνακες X και Y ώστε να είναι σε μορφή διανύσματος γραμμής ώστε κάθε γραμμή να είναι 1 input. Παρόμοια πράττω για τις τιμές εξόδου που τις εκχωρώ στο διάνυσμα T και θα αντιστοιχεί στο διάνυσμα των targets.

Εκτελώ την εντολή nrtool για να ανοίξει το γραφικό περιβάλλον εισάγω στα input τον πίνακα P και στα targets τον πίνακα T;

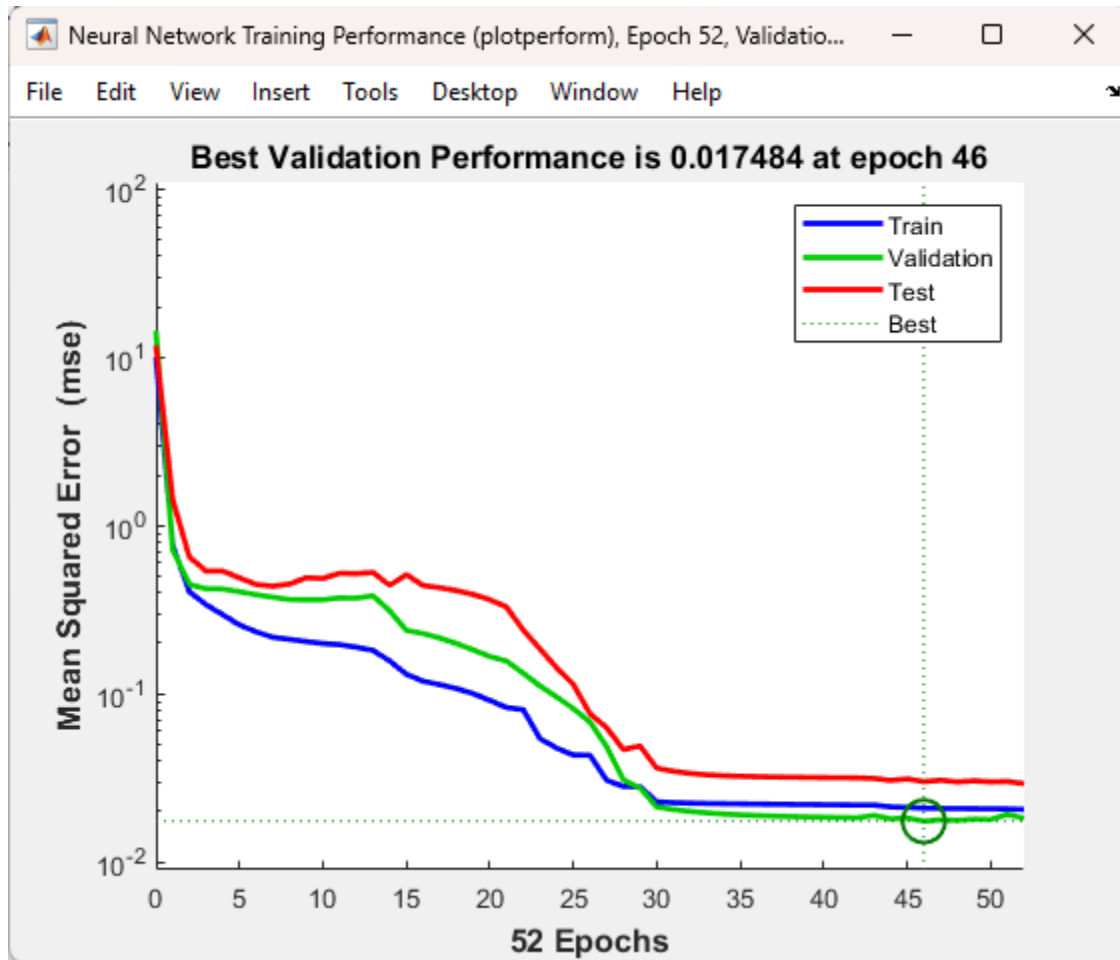
Η διαχώριση των δεδομένων θα γίνει ως εξής:

- 70% Train Set
- 15% Validation Set
- 15% Test Set

Μετά από δοκιμές, βρέθηκε ότι ο αλγόριθμος εκπαίδευσης Levenberg-Marquardt. Τα αποτελέσματα της εκπαίδευσης φαίνονται στην παρακάτω εικόνα:



Και το performance είναι το παρακάτω:



2. ΠΑΡΑΓΩΓΗ ΤΗΣ ΤΟΥ NPRTOOL ΜΕ ΚΩΔΙΚΑ

Το nftool δίνει την δυνατότητα να εξάγουμε το μοντέλο για να το τεστάρουμε, αλλά αντ' αυτού γράφτηκε ένα script που δημιουργήθηκε ένα νευρωνικό δίκτυο με τα ίδια χαρακτηριστικά. Ο κώδικας δίνεται στο επόμενο στιγμιότυπο:

```
1 clear; clc; close all;
2
3 x = linspace(-4,4,20); % -4:0.421:4; % Διάστημα μεταβλητής x.
4 y = linspace(-4,4,20); % -3.999:0.421:4; % Διάστημα μεταβλητής y.
5 [X, Y] = meshgrid(x, y); % Επίπεδο μεταβλητών x και y.
6 F = 0.7.*exp(cos(pi.*X))+0.3.*cos(2*pi.*Y); % Η Συνάρτηση που πρέπει να προσεγγίσουμε.
7
8 x = reshape(X, [1 numel(X)]); % Μετατροπή των x, y και της συνάρτησης σε διανύσματα
9 y = reshape(Y, [1 numel(Y)]); % για να τα χρησιμοποιήσουμε ως inputs και target διανύσματα
10 f = reshape(F, [1 numel(F)]); % ώστε να κάνουμε train το νευρωνικό δίκτυο.
11
12 clearvars X Y F;
13
14 P = [x;y]; T = f; % P -> Input, T-> Target
15
16 net=newff(minmax(P), [35, 1], {'tansig','purelin'},'trainlm');
17 net.numinputs = 1; % Αριθμός Εισόδων
18 net = configure(net,P);
19
20 %view(net)
21
22 net.divideFcn = 'dividerand'; % Διαχωρισμός δειγμάτων με τυχαίο τρόπο
23 net.divideParam.trainRatio = 0.7; % Train set
24 net.divideParam.valRatio = 0.15; % Validation set
25 net.divideParam.testRatio = 0.15; % Test set
```

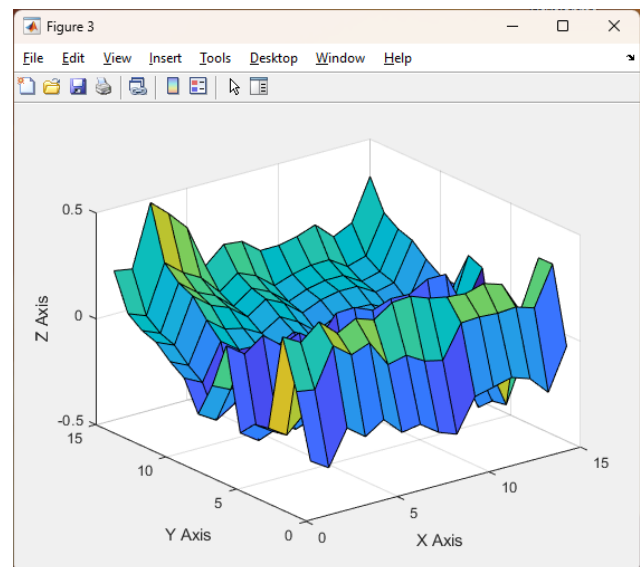
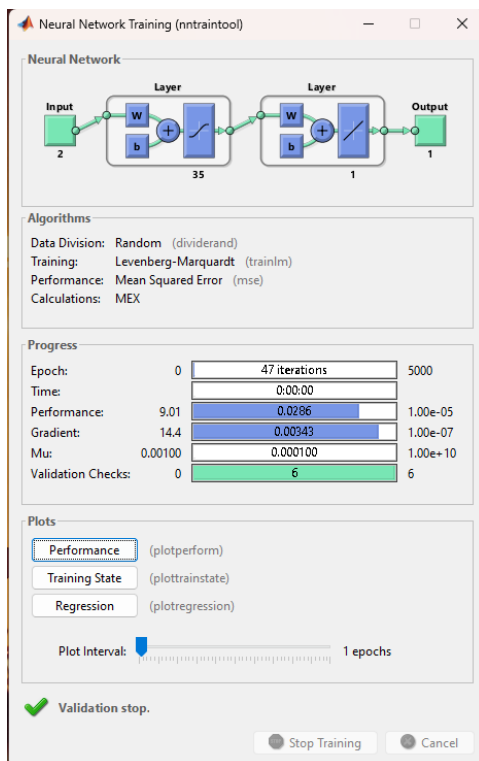
```

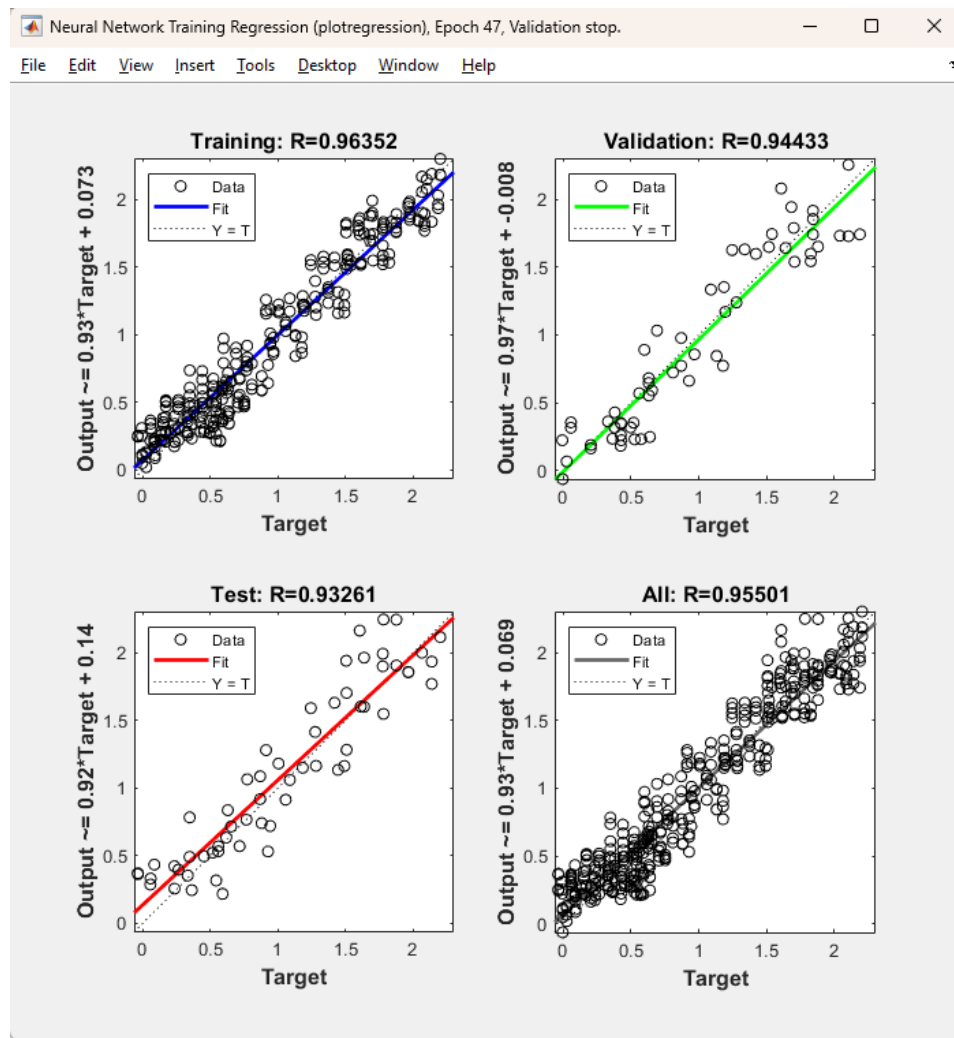
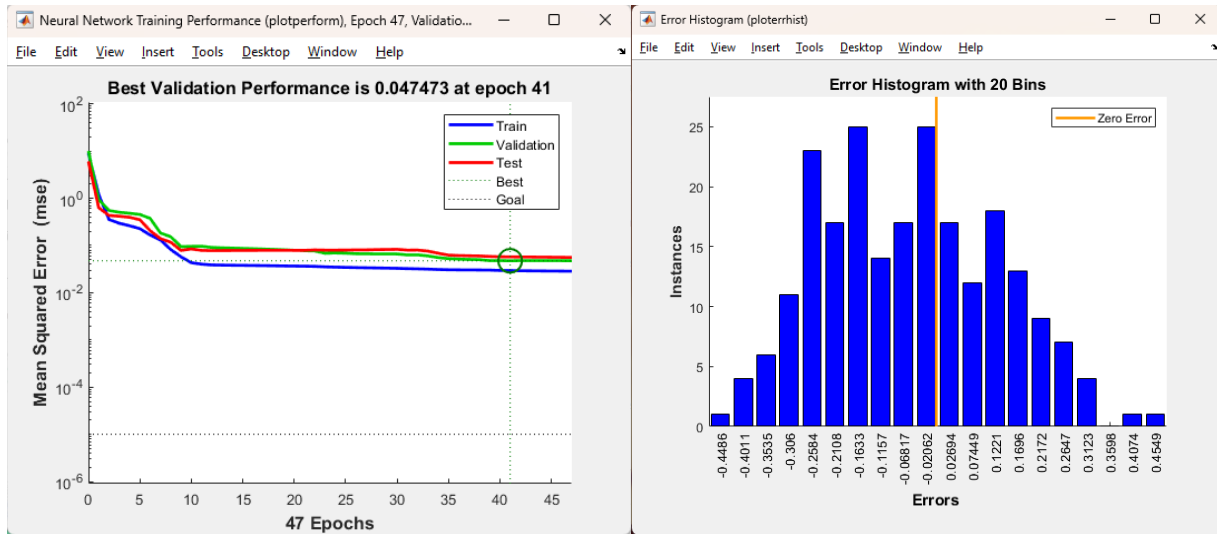
26
27 net.performFcn = 'mse'; % Mean Squared Error
28
29 net.trainParam.show = 50; %epochs between displays
30 net.trainParam.epochs = 1000; %maximum number of epochs to train
31 net.trainParam.goal = 1e-5; %performance goal
32
33 net = train(net, P, T);
34
35 f_out = net(P);
36
37 axlbl = @(h) [xlabel(h, 'X Axis'); ylabel(h, 'Y Axis'); zlabel(h, 'Z Axis')]; % Ετικέτες αξόνων στα τρισδιάστατα διαγράμματα.
38
39 x_test = sort(randi([-40000,40000],1,20)./10000);
40 y_test = sort(randi([-40000,40000],1,20)./10000);
41 [X_TEST, Y_TEST] = meshgrid(x_test, y_test);
42 fTest = 0.7.*exp(cos(pi.*X_TEST))+0.3.*cos(2*pi.*Y_TEST);
43 surf(fTest);
44 x_sim = reshape(X_TEST, [1, numel(X_TEST)]);
45 y_sim = reshape(Y_TEST, [1, numel(Y_TEST)]);
46
47 atest= sim(net,[x_sim; y_sim]);
48 atest = reshape(atest, [20 20]);
49 figure
50 surf(atest)
51
52 error = fTest- atest;
53 figure
54 surf(error);
55 perce = [mean(mean(error./fTest)), max(max(error./fTest))];
56 perf = perform(net, P, T);

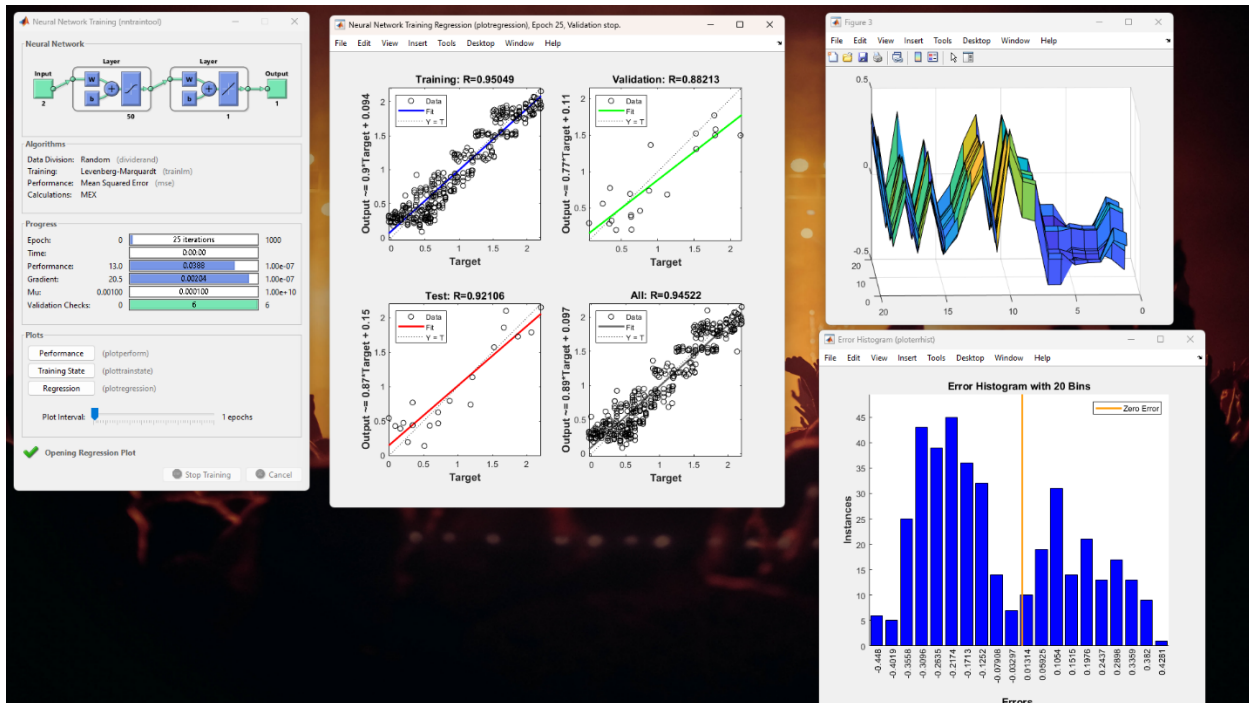
```

Στις γραμμές 39-54 παράγονται δύο τυχαία διανύσματα για τους άξονες x και y (x_test και y_test αντίστοιχα) τα οποία είναι ταξινομημένα με αύξουσα σειρά και χρησιμοποιούνται για να τυπώσουμε ένα διάγραμμα της πραγματικής συνάρτησης για αυτές τις τιμές των x και y, ένα διάγραμμα του εκπαιδευμένου μοντέλου-συνάρτησης και ένα διάγραμμα της διαφοράς αυτών των δύο, που στην ουσία δείχνει το σφάλμα.

Αφού εκτελεστεί ο κώδικας, τα αποτελέσματα της εκπαίδευσης είναι τα παρακάτω:

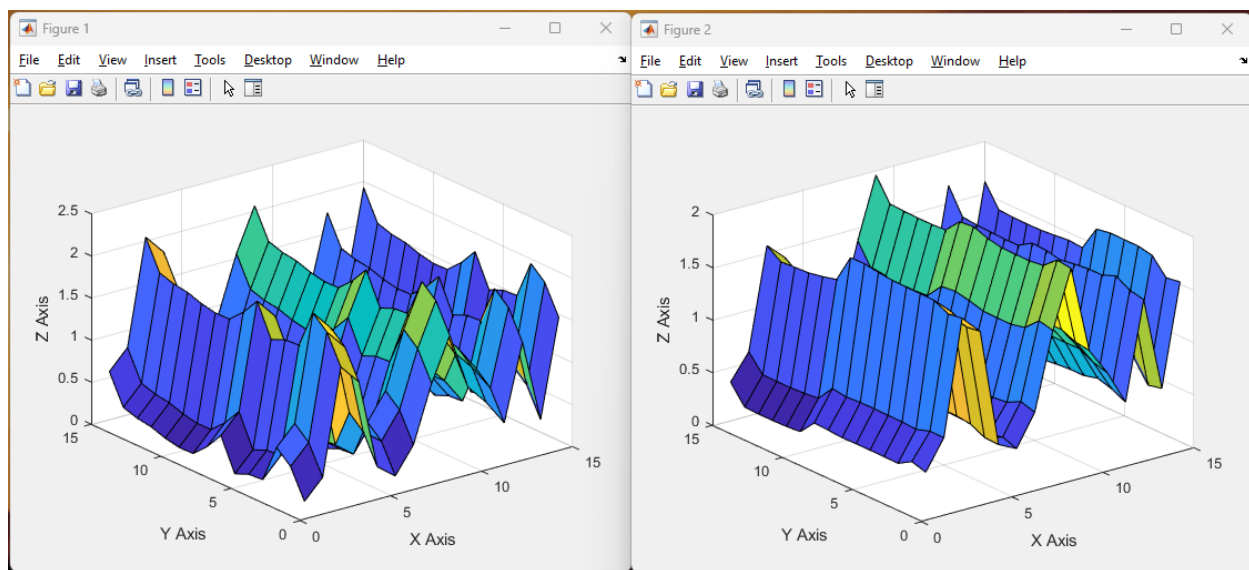


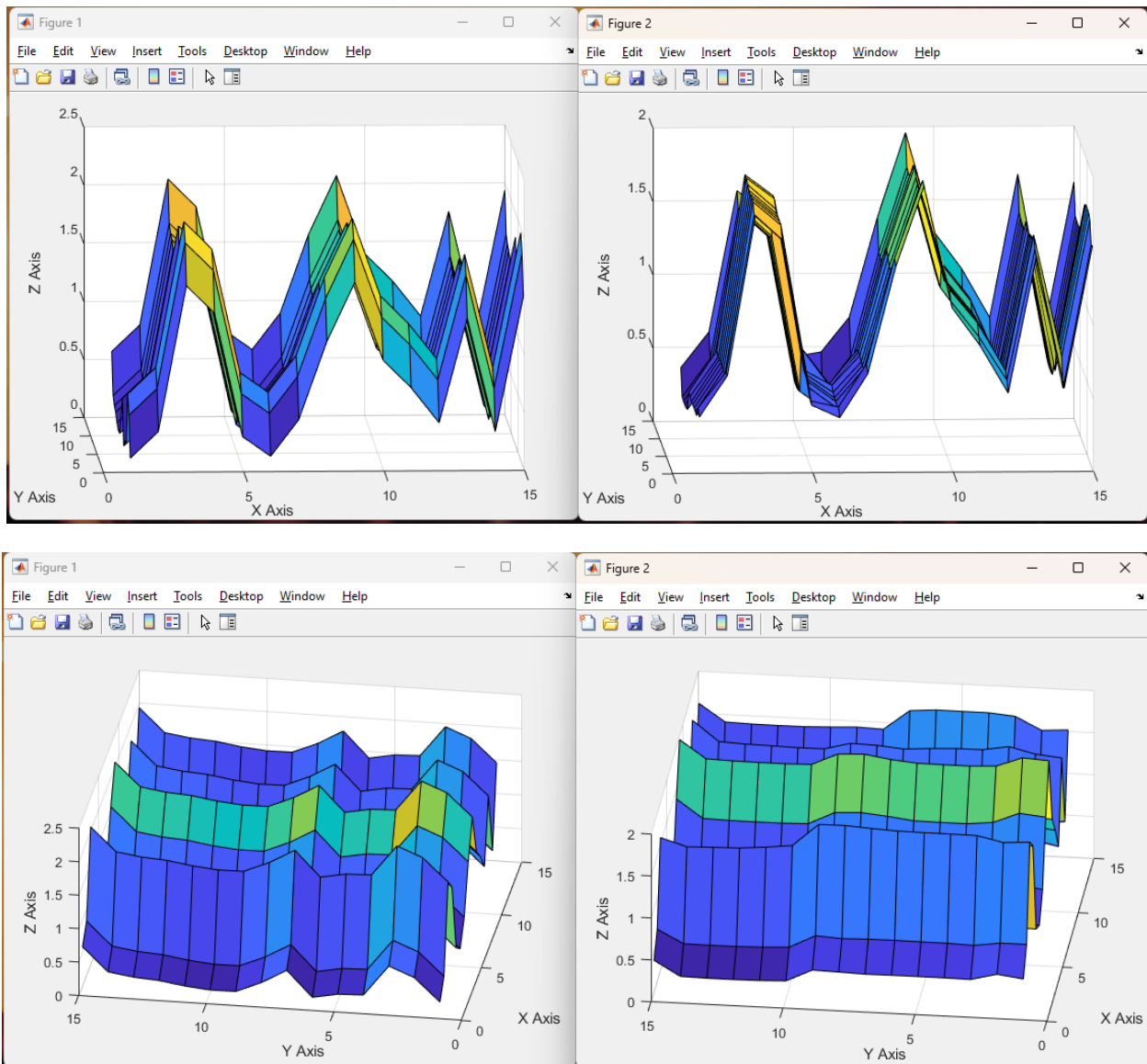




Το διάγραμμα πάνω δεξιά (Figure 3) δείχνει το σφάλμα για κάθε σημείο του επιπέδου μεταξύ της πραγματικής συνάρτησης και του εκπαιδευμένου μοντέλου. Σε ιδανικές συνθήκες θα επιθυμούσαμε να είναι ίσιο επίπεδο με $z = 0$ για κάθε τιμή των x και y .

Τα διαγράμματα της πραγματικής συνάρτησης (figure 1) και του εκπαιδευμένου δικτύου (figure 2) δίνονται παρακάτω από αρκετές οπτικές γωνίες:





Βλέπουμε ότι παρά την 92% επιτυχία προσέγγισης στα δεδομένα του Test set (αναφέρομαι σε αυτά γιατί θεωρητικά αυτά είναι τα άγνωστα δεδομένα που δόθηκαν στο μοντέλο), στην πραγματικότητα η συνάρτηση που εκπαιδεύτηκε από το νευρωνικό δίκτυο δεν είναι και πανομοιότυπη με αυτή που επιθυμούσαμε να πετύχουμε.

Το βασικό πρόβλημα που φαίνεται στις εικόνες είναι πως υπάρχει αντιστοίχιση το ενός άξονα (συγκεκριμένα του x) με τον z, ενώ η αντιστοίχιση του y με τον z δεν γινόταν πολύ καλά και φαινόταν ένα μικρό αποτέλεσμα στις μεγάλες κυματώσεις, αλλά και εκεί σε μικρό βαθμό.

Με άλλα λόγια όσο διατρέχουμε τον άξονα του x βλέπουμε τις ανόδους και τις καθόδους που ήταν αναμενόμενες με κάποιο σφάλμα, αλλά όσο διατρέχουμε τον άξονα του y τα αποτελέσματα δεν είναι το ίδιο καλά.

3.ΤΕΛΙΚΟ ΤΝΔ ΜΕ ΚΩΔΙΚΑ ΜΑΤΛΑΒ

Ο τελικός κώδικας που γράφηκε στο MATLAB βελτιώνοντας αισθητά την έξοδο του νευρωνικού δικτύου είναι ο παρακάτω:

```

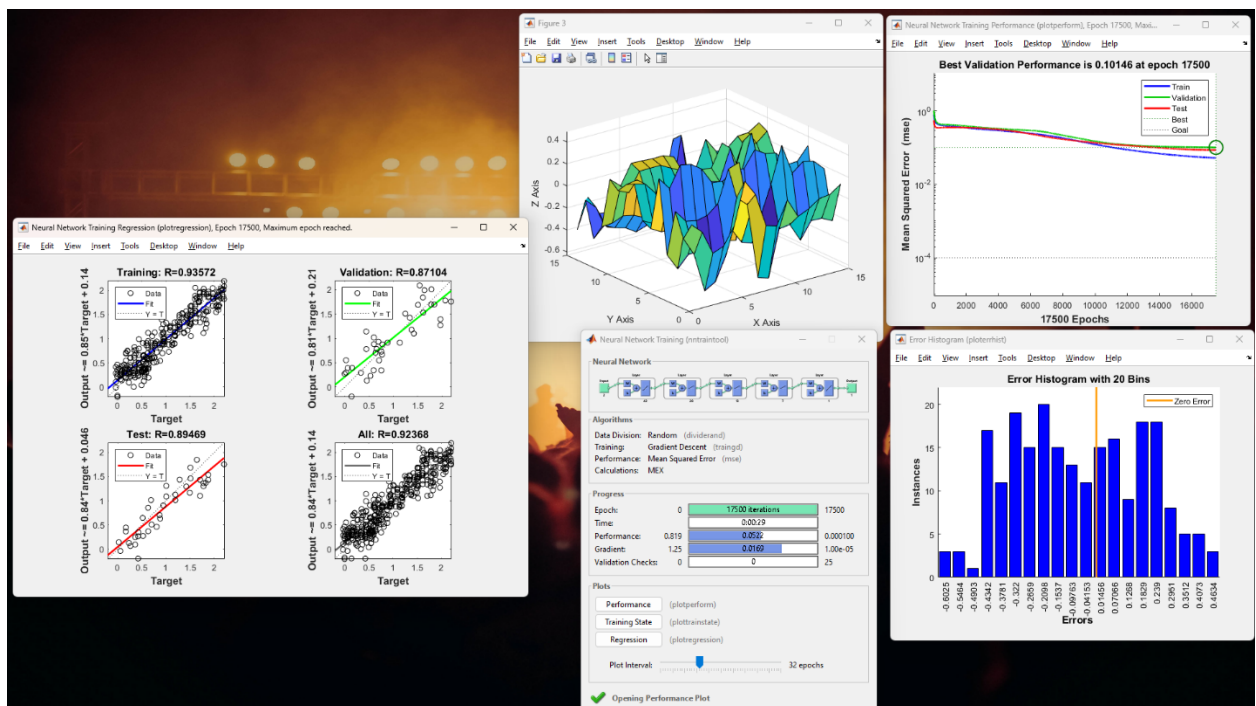
1 - clear; clc; close all;
2
3 - x = linspace(-4,4,20); % -4:0.421:4; % Διάστημα μεταβλητής x.
4 - y = linspace(-4,4,20); % -3.999:0.421:4; % Διάστημα μεταβλητής y.
5 - [X, Y] = meshgrid(x, y); % Επίπεδο μεταβλητών x και y.
6 - F = 0.7.*exp(cos(pi.*X))+0.3.*cos(2*pi.*Y); % Η Συνάρτηση που πρέπει να προσεγγίσουμε.
7
8 - x = reshape(X, [1 numel(X)]); % Μετατροπή των x, y και της συνάρτησης σε διανύσματα
9 - y = reshape(Y, [1 numel(Y)]); % για να τα χρησιμοποιήσουμε ως inputs και target διανύσματα
10 - f = reshape(F, [1 numel(F)]); % ώστε να κάνουμε train το νευρωνικό δίκτυο.
11
12 - clearvars X Y F;
13
14 - P = [x;y]; T = f; % P -> Input, T-> Target
15
16 - net=newff(minmax(P), [42, 28, 13, 7], {'tansig','logsig','logsig','logsig','purelin'},'traingd');
17 - net.numinputs = 1; %Αριθμός Εισόδων
18 - net = configure(net,P);
19
20 - %view(net)
21
22 - net.divideFcn = 'dividerand'; % Διαχωρισμός δειγμάτων με τυχαίο τρόπο
23 - net.divideParam.trainRatio = 0.8; % Train set
24 - net.divideParam.valRatio = 0.10; % Validation set
25 - net.divideParam.testRatio = 0.10; % Test set
26
27 - net.performFcn = 'mse'; % Mean Squared Error
28
29 - net.trainParam.show = 50; %epochs between displays
30 - net.trainParam.lr = 0.01; %learning rate
31 - net.trainParam.epochs = 17500; %maximum number of epochs to train
32 - net.trainParam.goal = 1e-4; %performance goal
33 - net.trainParam.max_fail = 25;
34
35 - net = train(net, P, T);
36
37 - f_out = net(P);
38
39 - axlbl = @(h) [xlabel(h, 'X Axis'); ylabel(h, 'Y Axis'); zlabel(h, 'Z Axis')]; % Ετικέτες αξόνων στα τρισδιάστατα διαγράμματα.
40
41 - x_test = sort(randi([-40000,40000],1,15)./10000);
42 - y_test = sort(randi([-40000,40000],1,15)./10000);
43 - [X_TEST, Y_TEST] = meshgrid(x_test, y_test);
44 - fTest = 0.7.*exp(cos(pi.*X_TEST))+0.3.*cos(2*pi.*Y_TEST);
45 - surf(fTest)
46 - axlbl(gca);
47
48 - x_sim = reshape(X_TEST, [1, numel(X_TEST)]);
49 - y_sim = reshape(Y_TEST, [1, numel(Y_TEST)]);
50
51 - clearvars x_test y_test X_TEST Y_TEST
52
53 - atest= sim(net,[x_sim; y_sim]);
54 - atest = reshape(atest, [15 15]);
55 - figure
56 - surf(atest)
57 - axlbl(gca);
58
59 - absError = fTest- atest;
60 - error = abs(absError/fTest);
61 - figure
62 - surf(absError)
63 - axlbl(gca);
64 - perce = mean(trimmean(absError, 1));
65 - figure
66 - ploterrhist(fTest-atest)
67 - perf = perform(net, P, T);
68

```

Οι αλλαγές που έγιναν σε σχέση με το αρχικό νευρωνικό δίκτυο, στην ουσία, είναι:

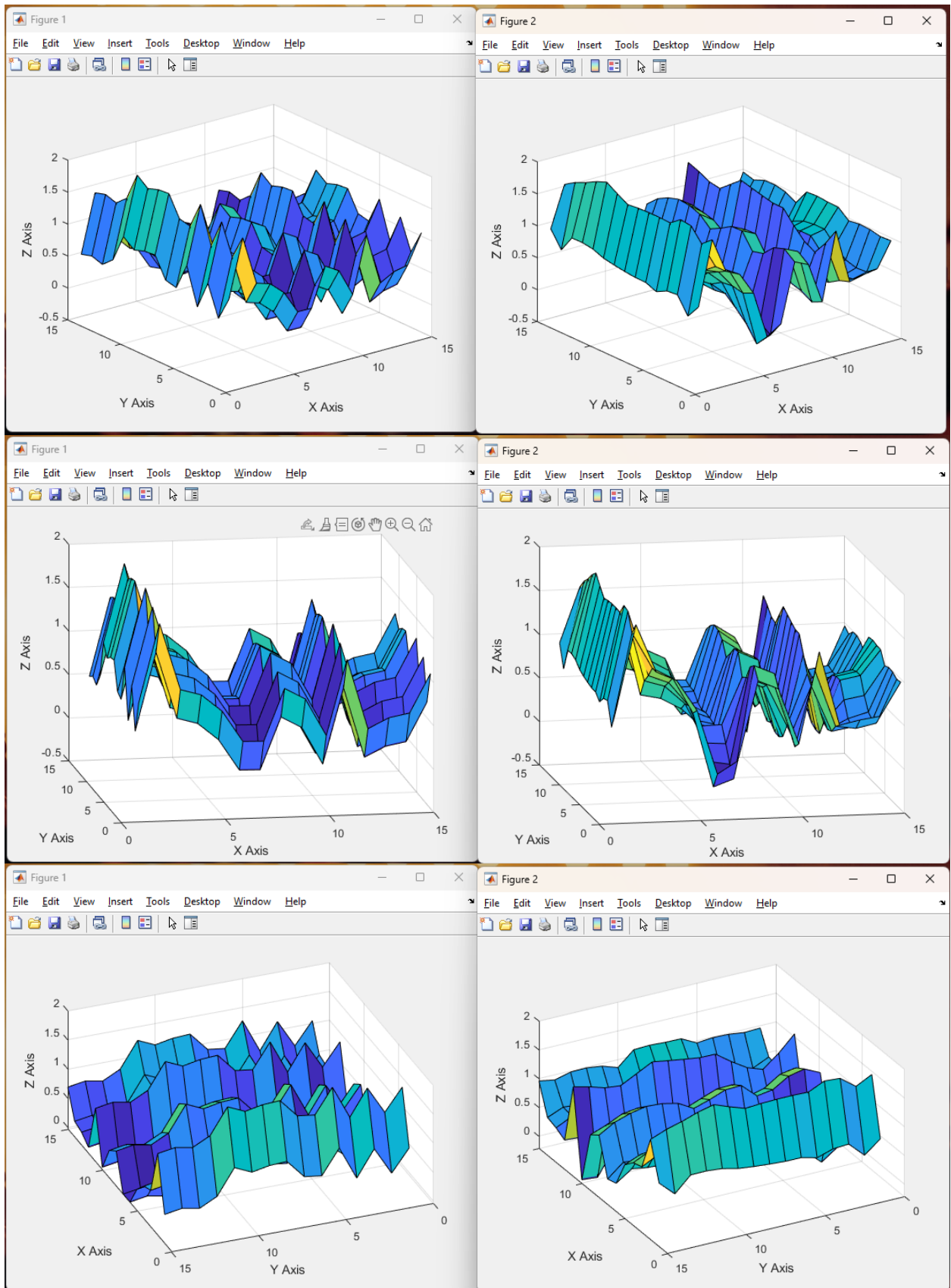
- Έχει συνολικά 5 hidden layers. Το πρώτο έχει 42 νευρώνες, το δεύτερο 28 νευρώνες, το τρίτο 13, το τέταρτο 7 και το τελευταίο έχει 1 νευρώνα, των οποίων οι συναρτήσεις ενεργοποίησης είναι τύπου 'logsig' (Λογαριθμική Σιγμοειδής), με εξαίρεση το πρώτο επίπεδο που έχει 'tansig', γιατί μετά από δοκιμές αποφάνθηκε ότι η εκπαίδευση επιτυγχάνει καλύτερο performance κατά μέσο όρο.
- Ο μέγιστος αριθμός έγινε 17500.
- Ο μέγιστος αριθμός συνεχόμενων αποτυχιών σε validation γίνεται 25.
- Ο διαχωρισμός των δεδομένων έγινε 0.8 - 0.1 - 0.1 .
- Η συνάρτηση εκπαίδευσης που χρησιμοποιήθηκε είναι η 'traingd'.

Τα αποτελέσματα μετά το train φαίνονται στα παρακάτω στιγμιότυπα:



Για να λειτουργήσει επαρκώς το μοντέλο πρέπει να τρέξουν και οι 17500 εποχές. Παρ' όλο που το τελικό performance έχει μικρότερη ακρίβεια από το προηγούμενο TND, στην πράξη το τελικό διάγραμμα είναι πιο κοντά ποιοτικά από το προηγούμενο. Ακόμη το Figure 3 της παραπάνω εικόνας δείχνει το σφάλμα που υπάρχει μεταξύ της πραγματικής συνάρτησης και του μοντέλου που είναι από -0.6 μέχρι 0.4.

Τα διαγράμματα της πραγματικής συνάρτησης και του μοντέλου φαίνονται παρακάτω:



Το μέσο σφάλμα κατά μέσο όρο ανά σημείο είναι -0.072.

 perce -0.0720

Ο υπολογισμός γίνεται με την εντολή στην γραμμή 64:

```
>> perce = mean(trimmean(absError, 1));
```

Η trimmean υπολογίζει τον μέσο όρο κάθε στήλης αφαιρώντας όμως τις πολύ ακραίες τιμές.

Γ. ΣΗΜΕΙΩΣΕΙΣ

Ο λόγος που δεν χρησιμοποιήθηκε η συνάρτηση Levenberg-Marquardt στην εκπαίδευση του είναι ότι σε μεγάλο αριθμό εποχών παρουσιάζει εξαιρετικά μεγάλο overfitting πολύ σύντομα.

Αν τα δείγματα αυξηθούν από 20 σε 30 ανά άξονα τα δείγματα γίνονται 900 και αν εκπαιδευτεί το δίκτυο για περισσότερες εποχές δεν παρουσιάζεται τόσο σύντομα overfitting οπότε βελτιώνεται η απόδοση του δικτύου.

